

# CCT College Dublin

## Assessment Cover Page

---

|                             |                        |
|-----------------------------|------------------------|
| <b>Module Title:</b>        | Databases              |
| <b>Assessment Title:</b>    | Data base construction |
| <b>Lecturer Name:</b>       | Marina Iantorno        |
| <b>Student Full Name:</b>   | James Scott            |
| <b>Student Number:</b>      | Sba23056               |
| <b>Assessment Due Date:</b> | 18/02/202              |
| <b>Date of Submission:</b>  | 18/02/202              |

---

### Declaration

By submitting this assessment, I confirm that I have read the CCT policy on Academic Misconduct and understand the implications of submitting work that is not my own or does not appropriately reference material taken from a third party or other source. I declare it to be my own work and that all material from third parties has been appropriately referenced. I further confirm that this work has not previously been submitted for assessment by myself or someone else in CCT College Dublin or any other higher education institution.

# **Database Design Document**

## **1.0. Introduction:**

This document provides a comprehensive overview of the design and implementation of the "Ireflix" database, tailored to support the functionalities of the Ireflix platform. It encompasses detailed discussions on various aspects including the database schema, SQL commands utilized for table creation, strategies for data population, database modifications, execution of queries, and a concluding summary highlighting key insights and outcomes.

## **2.0. Database Design:**

The database design encompasses several tables with specific attributes and constraints:

### **Users:**

- Username (VARCHAR (35))
- Name (VARCHAR (35))
- Surname (VARCHAR (35))
- Password (VARCHAR (15))

### **Movies:**

- ID\_movie (INT)
- Name (VARCHAR (35))
- Genre (VARCHAR (20))
- Duration (VARCHAR (10))
- Qualification (INT)
- Protagonist\_actor\_name (VARCHAR (35))
- Protagonist\_actor\_surname (VARCHAR (35))

**Series:**

- ID\_series (INT)
- Genre (VARCHAR (20))
- Qualification (INT)
- Protagonist\_actor\_name (VARCHAR (35))
- Protagonist\_actor\_surname (VARCHAR (35))
- Number\_of\_seasons (INT)

**Actors:**

- Name (VARCHAR (35))
- Surname (VARCHAR (35))
- Nationality (VARCHAR (50))

**Directors:**

- Name (VARCHAR (35))
- Surname (VARCHAR (35))
- Nationality (VARCHAR (50))

**MoviesActivity:**

- Username (VARCHAR (35)) [Foreign Key referencing Users (Username)]
- ID\_movie (INT) [Foreign Key referencing Movies (ID\_movie)]
- Genre (VARCHAR (20))

**SeriesActivity:**

- Username (VARCHAR (35)) [Foreign Key referencing Users (Username)]
- ID\_series (INT) [Foreign Key referencing Series (ID\_series)]
- Genre (VARCHAR (20))

### **3.0. Data Population:**

To populate the tables, a comprehensive approach was employed, utilizing both Python and Java scripts to gather data from diverse sources such as TMDb. Initially, an automated process was implemented, generating 500 unique user profiles with distinct usernames, surnames, and passwords.

Subsequently, the data population strategy evolved to incorporate a hybrid methodology. While automated scripts were utilized to collect and structure the initial dataset of movies and series, a manual insertion approach was adopted to ensure precision and integrity across all entries. This manual process involved meticulous curation and insertion of movie and series details, ensuring a comprehensive representation of recent popular releases from the past decade alongside timeless classics.

Furthermore, to simulate user activity within the platform Specifically, for MovierCctivity, SeriesActivity table, a SQL script was executed to insert data into these Tables. This script facilitated the creation of simulated user engagement by associating random series and movies IDs and corresponding genres with each user.

By amalgamating automated data retrieval with manual curation and simulated user interactions, the database was enriched with a diverse range of content and user engagement scenarios. This approach ensured the creation of a robust and comprehensive dataset, priming the database for seamless integration with the Ireflix database.

#### **4.0. Database Modification:**

##### **Addition of DirectorID Column:**

The Movies and Series tables were extended to include a new column named DirectorID, enabling the association of movies and series with their respective directors.

##### **Display Movies and Directors:**

SQL queries were executed to showcase the list of movies alongside their corresponding IDs and directors along with their IDs.

- **Update Movies and Series with DirectorID:**

The Movies and Series tables were updated to assign DirectorID to specific entries, thereby establishing relationships between movies/series and directors.

- **Addition of Year Column:**

Another enhancement involved appending a Year column to both the Movies and Series tables, facilitating the storage of release years for movies and series.

- **Insertion of Release Years:**

The Movies and Series tables were further updated to include release years for specific entries, enabling the organization and retrieval of data based on release timelines.

- **Display Actors:**

Additionally, a SQL query was executed to display the list of actors along with their corresponding IDs.

These modifications enrich the database schema, enabling comprehensive data management and retrieval capabilities related to movies, series, directors, and actors.

## **5.0. Query Execution:**

Various SQL queries were executed to retrieve specific information from the database. Examples include finding the number of movies a famous actor participated in and obtaining a list of actors who were protagonists in movies but not in series. Additionally, the queries encompassed:

- **Counting Movies Featuring a Famous Actor:**

This query aimed to determine the number of movies in which a renowned actor participated as a protagonist. It provided insights into the actor's prolificacy and popularity in the film industry.

- **List of Actors as Movie Protagonists but Not in Series:**

Another query focused on retrieving a list of actors who played protagonist roles in movies but did not appear in any series. This information could be valuable for casting decisions and identifying actors with diverse roles in different mediums.

These queries exemplify the versatility of SQL in extracting pertinent information from the database, aiding in decision-making processes related to casting, content analysis, and audience targeting.

## **6.0. Conclusion:**

The "Ireflix" database design was seamlessly executed, fulfilling all outlined requirements. Leveraging Python scripts to harness data from the TMDB API expedited the population process. Despite the initial automated data collection, meticulous manual insertion was undertaken across all tables to ensure accuracy and completeness. Subsequent database modifications were adeptly performed, enhancing its functionality and readiness for integration with Ireflix database. In conclusion, the project culminated in a resilient database structure primed to support Ireflix database effectively.